Unit IV

Number System and Logic Gates (7 hrs)

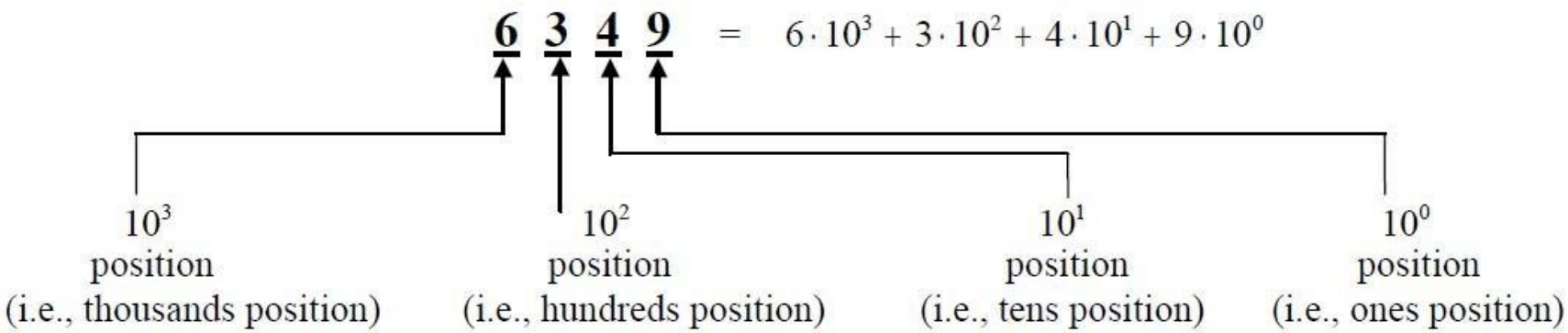
Syllabus:

- Number System:-
  - Binary, BCD, Octal, Decimal, Hexadecimal their conversion and arithmetic
  - De-Morgan's theorem.
- Basic Gates:-
  - AND, OR, NOT
- Universal Gate-
  - NAND, NOR
- Exclusive Gates
  - XOR, XNOR
- Combinational Design
  - Half adder, Full adder
- Flip Flop's
  - SR, JK, T and D
- Introduction to Microprocessor and Microcontroller

Number System

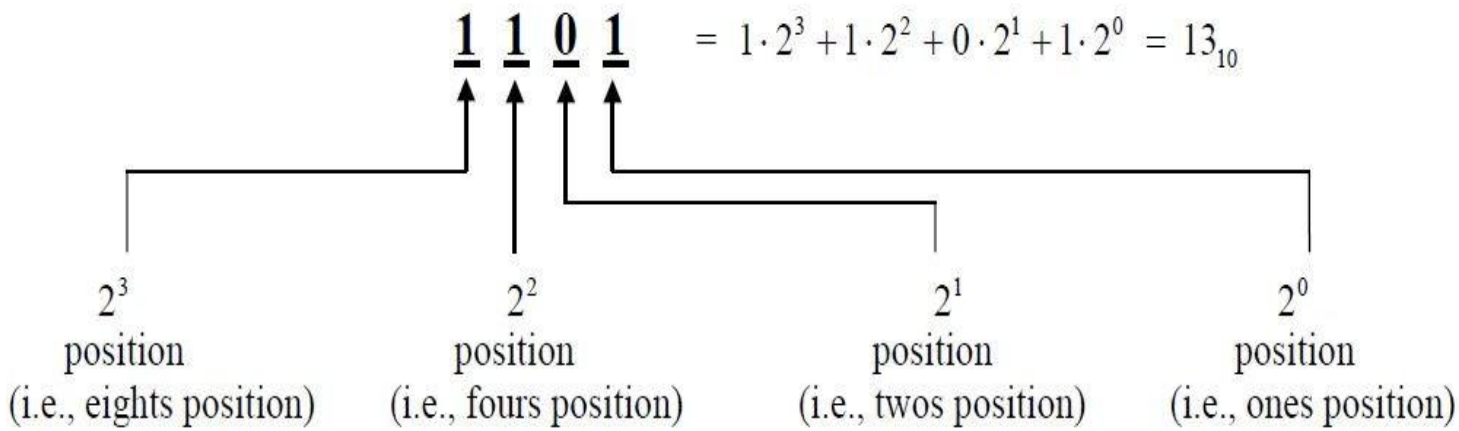
•Decimal

- The number system we are familiar with, used every day, is the *decimal number system.*
- Referred to as the **base-10 s**ystem.
- The base-10 system is a *positional system*
- The base-10 number system has 10 distinct symbols, or digits **(0, 1, 2, 3,…8, 9).**



Binary

- The point: All data in a computer is represented in binary. A base-2 system, or *binary number system.*
- The term 'bit' is a contraction of the words 'binary' and 'digit', and when talking about binary numbers.
- The **base-2** system has exactly two symbols: 0 and 1.
- 0 and 1 are *logical values, not the values of a physical quantity.*
- *A string of eight bits (such as 11000110) is termed a byte. A collection of four bits (such as 1011) is smaller than a byte, and is*



Fractional binary to Fractional decimal conversion

- Write given fractional binary number
- Multiply each bit by corresponding positional weight of bit
- Find sum of product . It gives decimal fractional equivalent of fractional binary number
- Example $(10111.1011)_2 = (\ \ )_{10}$

1 0 1 1 1 . 1 0 1 1

$1x\ 2^4 + 0x\ 2^3 + 1x\ 2^2 + 1x\ 2^1 + 1x\ 2^0 .\ 1x\ 2^{-1} + 0x\ 2^{-2} + 1x\ 2^{-3} + 1x\ 2^{-4}$

16+0+4+2+1 . 0.5+0+0.125+0.0625

$(23.6875)_{10}$

Fractional Decimal to Fractional Binary conversion

1. Write given fractional decimal number
2. Multiply fractional decimal number by 2 and write carry integer part . First carry is MSB
3. Multiply fractional part of product obtained in step2 and note down the carry.
4. Repat step 3 till the fractional part of product is zero.
5. Last carry is LSB

- Example $(0.33)_{10} = (\ \ )_2$

Decimal fraction	BaseX2	Product	Fractional Part	Carry(integer part)
0.33	X 2	0.66	0.66	0 (MSB)
0.66	X 2	1.32	0.32	1
0.32	X 2	0.64	0.64	0
0.64	X 2	1.28	0.28	1
0.28	X 2	0.56	0.56	0 (LSB)

Octal number system and Octal to decimal conversion

- Octal Numbering System Eight symbols:
  - 0, 1, 2, 3, 4, 5, 6, 7 •
  - Notice that we no longer use 8 or 9 Base Comparison: Base 10:

0, 1, 2, 3, 4, 5, 6, 7, 8 , 9, 10, 11, 12…

- Base 8: 0, 1, 2, 3, 4, 5, 6, 7, 10 , 11, 12, 13, 14… Example: What is

(15) 8 in base 10?

- (15.28)8 = (1 x 8^1) + (5 x 8^ 0). 2x 8^-1+8x 8^-2 = (13.375)10
- Example: What is (7061)8 in base 10?
  - (7061)8 = (7 x 8 ^3) + (0 x 8^2) + (6 x 8^1) + (1 x 8^0)
    - = 36331
    - 1582 OCT = invalid octal number

Hexadecimal Number and Hexadecimal to decimal conversion

- The base-16 hexadecimal number system has 16 digits (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E and F).
- Converting between binary and hexadecimal is exceedingly easy— much easier than converting between binary and decimal
- (FF.2F)hex = $(\quad)_{10}$
- 15x16^1 + 15x16^0 . 2x16^-1+ Fx16^-2 = 240 +15=255+0.1835= 255.1835

Converting a Hexadecimal Number to a Decimal Number

- Write the hexadecimal number as a sum of powers of 16.
- For example, considering the hexadecimal number 1A9B above, we convert this to decimal as:
- 1C9D = 1x16^3+12x16^2+9x16^1+13x16^0
- =7325

BCD(Binary Coded Decimal) Number System

- In this system each digit is represented by 4 bit binary number.
- BCD is way to express each decimal digit with binary number.
- In BCD system only first ten decimal no. from 0 to 9 represented in four bit binary equivalent number .(0000 to 1001)
- Remaining six (1010 to1111) are invalid 4bit BCD number
- It is also called 8-4-2-1 code because weight of code are 8-4-2-1 from left (MSB)to right(LSB).

BCD(Binary-Coded Decimal)

- In some practical applications we will use a digital logic circuit to drive a numeric display.
- individual display unit displays a single digit. So, for example, we might have the number 472
- logic circuit, and we would like to display this on three separate display units (one for the 4, one for 7 and one for the 2).
- Working with this sort of display hardware is facilitated through the use of binary-coded decimal (BCD),
- where each individual digit is represented by a 4-bit number. For example, to represent the decimal
- number 472 in binary-coded decimal (BCD), we convert each digit to a four bit binary number,
- independent of the other decimal digits. Thus, 472 equal
- 0100 0111 0010 in BCD.

Converting a Decimal Number to a Binary Number: Method 1

- The first method of converting from a decimal number to a binary number entails expressing the decimal number as a sum of powers of 2.
- To convert the decimal number *x to binary:*
  - Step 1. Find the highest power of two less than or equal to *x. The binary representation will have a one* in this position. Denote the value of this highest power of 2 as *y.*
  - Step 2. Now subtract this power of two (*y) from the decimal number (x), denoting the result as z:*
    - *Z = x - y.*
  - Step 3. If *z = 0, you are done. Otherwise, let x = z and return to Step 1 above.*

Converting a Decimal Number to a Binary Number: Method 2

- The second method of converting a decimal number to a binary number entails repeatedly dividing the decimal number by 2, keeping track of the remainder at each step.
- To convert the decimal number *x to* binary:
  - Step 1. Divide *x by 2 to obtain a quotient and remainder. The remainder will be 0 or 1.*
  - Step 2. If the quotient is zero, you are finished: Proceed to Step 3. Otherwise, go back to Step 1, assigning *x to be the value of the most-recent quotient from Step 1.*
  - Step 3. The sequence of remainders forms the binary representation of the number.
- This method of repeated division by 2 is called the 'double-dabble' method.

Convert $(26)_{10}$ into a Binary number.

Solution:

Division	Quotient	Generated remainder
$\frac{26}{2}$	13	0
$\frac{13}{2}$	6	1
$\frac{6}{2}$	3	0
$\frac{3}{2}$	1	1
$\frac{1}{2}$	0	1

Hence the converted binary number is $(11010)_2$.

Converting a Decimal Number to a Hexadecimal Number

- To convert from decimal to hexadecimal is to use the same division algorithm that you used to convert from decimal to binary, but repeatedly dividing by 16 instead of by 2.
- As before, we keep track of the remainders, and the sequence of remainders forms the hexadecimal representation.

  - Convert $(348)_{10}$ into a hexadecimal number.

Division	Quotient	Generated remainder
$\frac{348}{16}$	21	12
$\frac{21}{16}$	1	5
$\frac{1}{16}$	0	1

Hence the converted hexadecimal number is $(15C)_{16}$.

Converting a Hexadecimal Number to a Binary Number

- We can convert directly from hexadecimal notation to the equivalent binary representation by using the following procedure:
  - Convert each hexadecimal digit to a four digit binary number, independent of the other hexadecimal digits.
  - Concatenate the resulting four-bit binary numbers together.
  - For example, to convert the hexadecimal number 4DA9 to binary, we first convert each hexadecimal digit to a four-bit string:
    - 4 = 0100 D = 1101 A = 1010 9 = 1001
  - and then concatenate the results: The resulting binary number is: 0100 1101 1010 1001.
  - We can drop leading zeros (from the leftmost quartet only!), giving us:
    - 4DA9.B4 =0100 1101 1010 1001.1011 0100
    - FF.A1 = 1111 1111. 1010 0001
    - 0011 1001 = 39

Converting a Binary Number to a Hexadecimal Number

- Convert directly from binary notation to the equivalent hexadecimal
- representation by using the following procedure:
  - Starting at the right, collect the bits in groups of 4
  - Convert each group of 4 bits into the equivalent hexadecimal digit
  - Concatenate the resulting hexadecimal digits
  - For example, to convert 110110101001 to hexadecimal, we collect the bits into groups of 4 starting at the right: 1101 1010 1001, and then we convert each collection of bits into a hexadecimal digit:
  - 1101 1010 1001
  - D A 9
  - Thus 0001 1101 1010 1001 = 1DA9

- 176 OCT = 001 111 110
- 61 OCT = 110 001
- 011 111 = 3 7

Converting Binary to Hexadecimal

- Example
- (1001111100001 11.11100001111)
  (100.111.1100.0011.1.1110.0011.11.)$_2$ = (5FC3.E.1E.)$_{16}$
- **For Integer binary number** make group of four bits with starting from LSB of integer part.
- If, for last MSB digit group of four bits is not possible then append no. of zeros less to leftmost of bits .
- Write HEX equivalent digit for every set of four bits

↓(appended zero)

0100 1111 1000 0111

4 F C 3

Binary to Hexa Decimal conversion

- For fractional binary number make group of four bits with starting from MSB of fractional binary and
- If for last LSB group of four bits is not possible then append shorted no. of zeros to rightmost of bits.
- Example
- .1110 0001 111 (fractional binary number.)
- ↓(appended zero)
- 1110 0001 1110
- E 1 E
- .E1E

Number Representation

Decimal	Octal	Binary	Hex
0	0	0000	0
1	1	0001	1
2	2	0010	2
3	3	0011	3
4	4	0100	4
5	5	0101	5
6	6	0110	6
7	7	0111	7
8	10	1000	8
9	11	1001	9
10	12	1010	A
11	13	1011	B
12	14	1100	C
13	15	1101	D
14	16	1110	E
15	17	1111	F
16	20	0001 0000	10
17	21	0001 0001	11
18	22	0001 0010	12
19	23	0001 0011	13

8-4-2-1 System

MSB LSB

Decimal	2^3=8	2^2=4	2^1=2	2^0=1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
13	1	1	0	1
10	1	0	1	0

Dec 0= 0x2^3 + 0x2^2 + 0x2^1 + 0x2^0
=0

Logic Gates

- There are seven logic gates:
  - AND, OR, XOR, NOT, NAND, NOR, and XNOR.
- There are three **basic types** of digital logic gates
  - AND Gate
  - OR Gate and
  - NOT Gate (NOT gates and Buffers are single input devices that can also have a **Tri-state** High-impedance output which can be used to control the flow of data onto a common data bus wire)
- **Universal** gates
  - The NAND gate and the NOR gate both are classed as "**Universal**" gates as they can be used to construct any other gate type.
  - Any combinational circuit can be constructed using only two or three input NAND or NOR gates.
- Exclusive Gates
  - EX-OR
  - EX-NOR

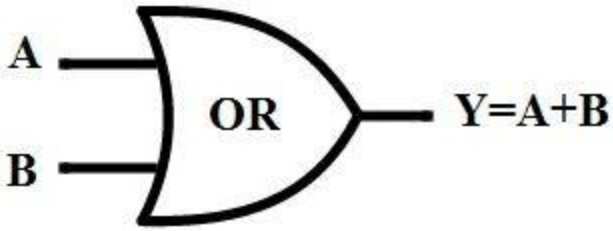
AND GATE

- The AND gate has two or more inputs and one output signal.
- This is a gate which performs a logical operation multiplication, which is commonly known as AND function.
- This gate is called an AND gate because the gate gives output will be high when all input values are high the output is low or 0 when any of the inputs are low or 0.
- The symbol of AND Gate & Truth Table is shown below.

Inputs		Output
A	B	Y=A.B
0	0	0
0	1	0
1	0	0
1	1	1

OR Gate

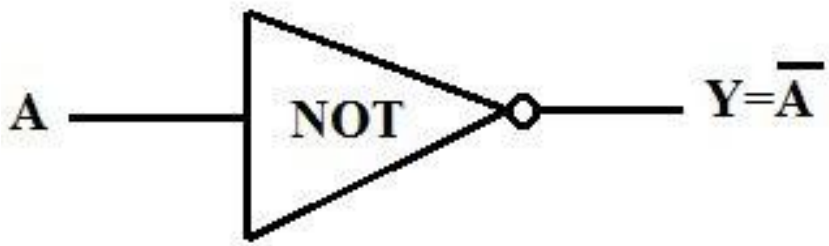
- The OR gate has two or more inputs and one output signal.
- A logical operation or addition, which is commonly known as OR function.
- This gate is called an OR gate because the gate gives output LOW only if any or all input values are LOWThe output is low or 0 when both the inputs are low or 0.
- The symbol of OR Gate & Truth Table is shown below.



Inputs		Output
A	B	Y=A+B
0	0	0
0	1	1
1	0	1
1	1	1

NOT GATE

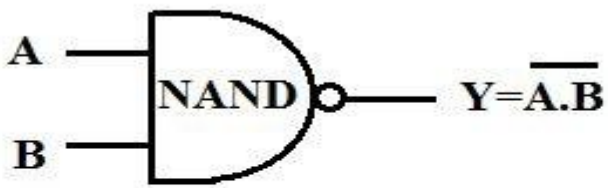
- Single input and a single output
- Known as an inverter, because it does the inversion of applied binary signal at the
- The symbol of NOT Gate & Truth Table is shown below.



Input	Output
A	$Y=\overline{A}$
0	1
1	0

NAND GATE

- Has two or more input terminals and one output terminals.
- Cascade combination of AND – NOT gates.
- The output of NAND gate is high or 1 when any one input is low or 0 and the output is low or 0 when all inputs are high or 1.
- The symbol of NAND Gate & Truth Table is shown below.

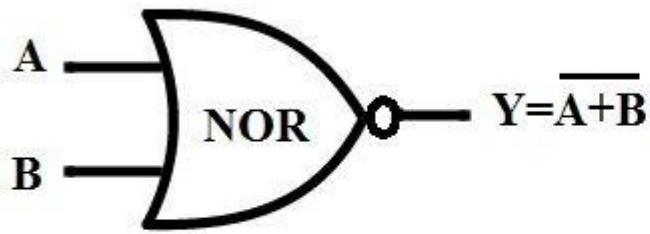


Inputs		Output
A	B	$Y=\overline{A.B}$
0	0	1
0	1	1
1	0	1
1	1	0

NOR GATE

- Has two or more input terminals and one output terminal.
- Cascade combination of OR – NOT gates.
- The output of NOR gate is high or 1 when all the inputs are low or 0 and the output is low or 0 when one or all inputs are high or 1.

The symbol of NOR Gate & Truth Table is shown below.



Inputs		Output
A	B	$Y=\overline{A+B}$
0	0	1
0	1	0
1	0	0
1	1	0

Exclusive- OR (EX-OR) GATE:

- This gate has two or more input terminals and one output terminal.
- The EX-OR Gate has the output only high when an odd number of inputs are high and the output is low when both the inputs are low and both the inputs are high.
- The symbol of EX-OR Gate & Truth Table is shown below.

$Y = A \oplus B$

$Y = \bar{A}B + A\bar{B}$

Inputs		Output	
A	B	$Y = A \oplus B$	
0	0	0	
0	1	1	$\bar{A}B$
1	0	1	$A\bar{B}$
1	1	0	

Exclusive- NOR (EX-NOR) GATE

- The EX-NOR gate has terminals like OR gate i.e, two or more input terminals and one output terminal.
- The EX-NOR Gate has the output only high when both the inputs have the same values either high or low and the output is low when an odd number inputs are high.
- The symbol of EX-NOR Gate & Truth Table is shown belc

$Y=\overline{A \oplus B}$

$Y=\overline{AB} + AB$

Inputs		Output	
A	B	$Y=\overline{A \oplus B}$	
0	0	1	$\overline{AB}$
0	1	0	
1	0	0	
1	1	1	AB

AND Laws	OR Laws
$A \cdot 1 = A$	$A + 1 = 1$
$A \cdot 0 = 0$	$A + 0 = A$
$A \cdot A = A$	$A + A = A$

DeMorgans Theorem

- De Morgan's most important work are two related theorems that have to do with how NOT gates are used in conjunction with AND and OR gates:
  - An AND gate with inverted output behaves the same as an OR gate with inverted inputs.

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$

  - An OR gate with inverted output behaves the same as an AND gate with inverted inputs.

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

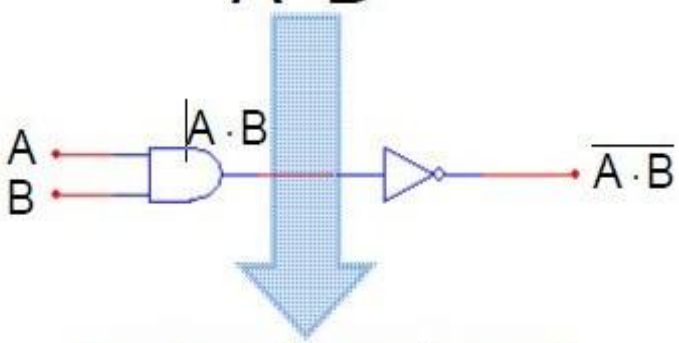
  - An AND gate with inverted output is also called a NAND gate, of course, and an OR gate with inverted output is also called a NOR gate.
- Thus, De Morgan's laws can also be stated like this:
  - A NAND gate behaves the same as an OR gate with inverted inputs.
  - A NOR gate behaves the same as an AND gate with inverted inputs.
- simplification techniques used to simplify Boolean expressions.
- the simpler the resulting the Boolean expression, the simpler the resulting logic.

DeMorgan's First Theorem

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$

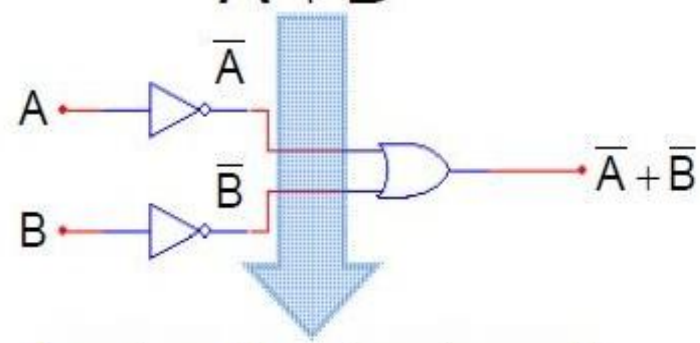
Proof

$\overline{A \cdot B}$



A	B	$A \cdot B$	$\overline{A \cdot B}$
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

$\overline{A} + \overline{B}$



A	B	$\overline{A}$	$\overline{B}$	$\overline{A} + \overline{B}$
0	0	1	1	1
0	1	1	0	1
1	0	0	1	1
1	1	0	0	0

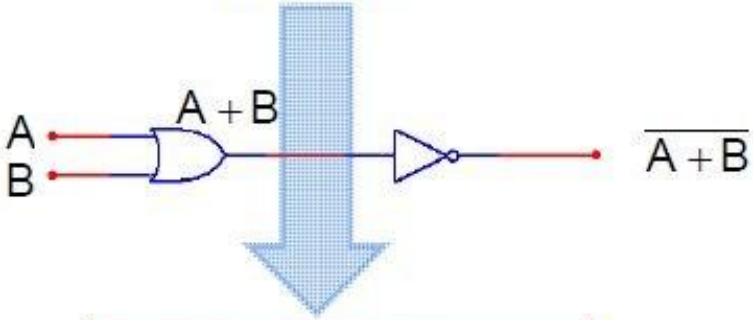
The truth-tables are equal; therefore, the Boolean equations must be equal.

DeMorgan's Second Theorem

$$\overline{A+B} = \overline{A} \cdot \overline{B}$$

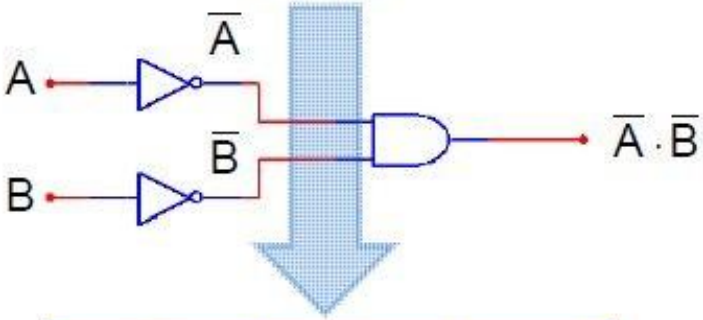
Proof

$\overline{A+B}$



A	B	A + B	$\overline{A+B}$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

$\overline{A} \cdot \overline{B}$



A	B	$\overline{A}$	$\overline{B}$	$\overline{A} \cdot \overline{B}$
0	0	1	1	1
0	1	1	0	0
1	0	0	1	0
1	1	0	0	0

The truth-tables are equal; therefore, the Boolean equations must be equal.

Arithmetic Operations

- **Rules of Binary Addition**

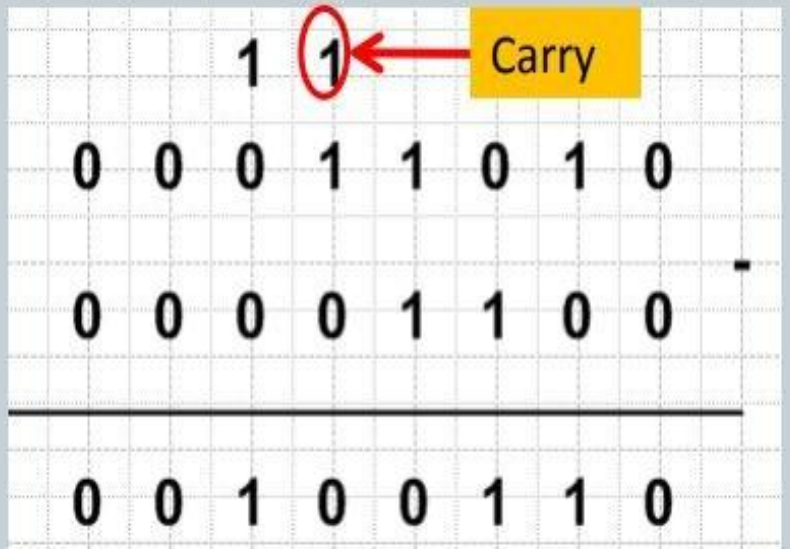
0 + 0 = 0

0 + 1 = 1

1 + 0 = 1

1 + 1 = 0, and carry 1 to the next more significant bit

- Example: 26 +12



Binary Addition

- Conceptually similar to decimal addition
- *Example:* Add the binary numbers 1010 and 11

(carry)

1

```
  1 0 1 0
+     1 1
---------
  1 1 0 1
```

Binary Subtraction

The four basic rules for subtracting are as follows:

0-0=0

1-1=0

1-0=1

0-1=1 0-1 with a borrow of 1

m 1011

(borrow)

$$
\begin{array}{rcccc}
 & 0 & 1 & & \\
 & 1 & \cancel{0} & 1 & 1 \\
- & & 1 & 0 & 1 \\
\hline
 & 0 & 1 & 1 & 0
\end{array}
$$

1's And 2's Complement of Binary Number

The 1's complement and the 2's complement of binary number are important because they permit the representation of negative numbers.

Binary Number	**1**	**0**	**1**	**1**	**0**	**0**	**1**	**0**
1'sComplement	**0**	**1**	**0**	**0**	**1**	**1**	**0**	**1**

2's Complement of a binary number is found by adding 1 to the LSB of the 1's Complement.

2's Complement= (1's Complement) +1

Binary number		10110010
1'scomplement		01001101
Add 1	+	1
2's complement		01001110

Subtraction by using 1's Complement:

Step 1: Convert number to be subtracted to it's 1's complement form.

Step 2: Perform the addition.

Step 3: If the final carry is 1, then add it to the result obtained in step 2. If final carry is 0, result obtained in step 2 is negative and in the 1's complement form.

Ex:- 2 Perform $(0101)_2 - (1100)_2$

A = 0101

B = 1100

-B = 0011

```
  1 1 1
  0 1 0 1  (5)10
+ 0 0 1 1  (-12)10 ve
-----------
  0 1 0 0 0  (-7)10
```

0111 = $(7)_{10}$

GROW

Example 1 =

$(1001)_2 - (0101)_2$

$X = (1001)_2$

$Y = (0101)_2$

Subtracted no. convert into 1's complement

1's complement $Y = 1010 = -5$

Add X AND Y

$$\begin{array}{r} 1001 \\ +1010 \\ \hline 10011 \end{array} \quad \begin{array}{l} (9)_{10} \\ (-5)_{10} \\ \\ (4)_{10} \end{array}$$

If carry 1 then 1 add in sum

If carry 0 result is –ve and 1's comp form

$$\begin{array}{r} 11 \\ 0011 \end{array}$$

$(0101)_2 - (1101)_2$

$X = (0101)_2 = 5$

$Y = (1101)_2 = 13$

1's complement - Y = 0010 = -13

$$\begin{array}{r} 0101 \\ +0010 \\ \hline 00111 \end{array}$$

LEARN

Binary Subtraction using Two's Complement

- Calculation of 2's Complement

- Step 1:
  - invert the binary equivalent of the number by changing all of the ones to zeroes and all of the zeroes to ones (also called 1's complement)
- Step 2:
  - Then add one.
- Example: -17

17 = 00010001

Step1 : 11101110

```
Step2 : 11101110
        +       1
-----------------
        11101111
```

-17 = 11101111

2's Complement Addition

- Two's complement addition follows the same rules as binary addition.
- Example: 5 + (-3)

5 = 00000101

-3 = 11111101

```
  00000101
+ 11111101
----------------------
1 00000010  (2)
```

Ignore

2's Complement Addition

- Two's complement addition follows the same rules as binary addition.
- Example: 3 + (-5)

3 = 00000011

-5 = 11111011

```
  00000011
+ 11111011
----------------------
 11111110  (-2)
```

Adding/Subtracting using 2's Complement

2s complement notation makes it possible to add and subtract signed numbers

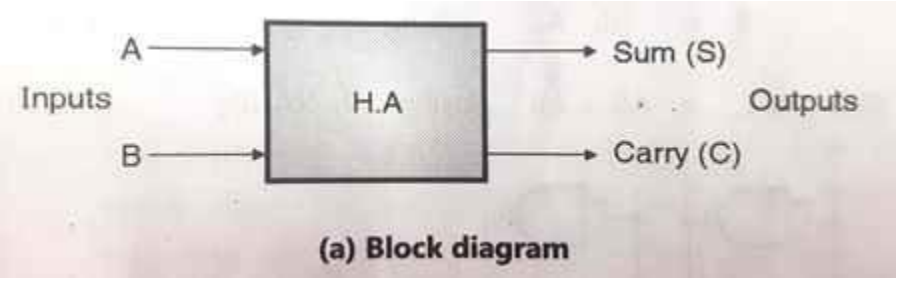
(Decimal)	2s Complement	
(- 1)	1 1 1 1	
+ (- 2)	+ 1 1 1 0	
(- 3)	$\boxed{1}$ 1 1 0 1	2s complement
	Discard	
(+1)	0 0 0 1	
+ (- 3)	+ 1 1 0 1	
(- 2)	1 1 1 0	2s complement

Adders

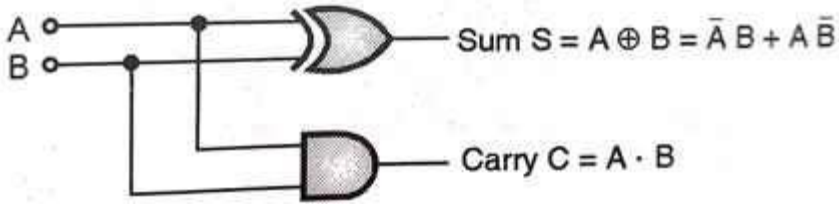
- The most basic operation a computer can perform is to add two numbers together through binary
- these types of addition operations are carried out by a special kind of combinational circuit called adders
- A combination circuit that performs the addition of two bits is **half adder**
- A adder performs the addition of 2 significant bits and a previous carry is called a **full adder**

Half Adder

- Half Adder: is a combinational circuit that performs the addition of two bits, this circuit needs two binary inputs and two binary outputs.



(a) Block diagram



(C) Logic Diagram

Inputs		Outputs	
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$S = \bar{A}B$

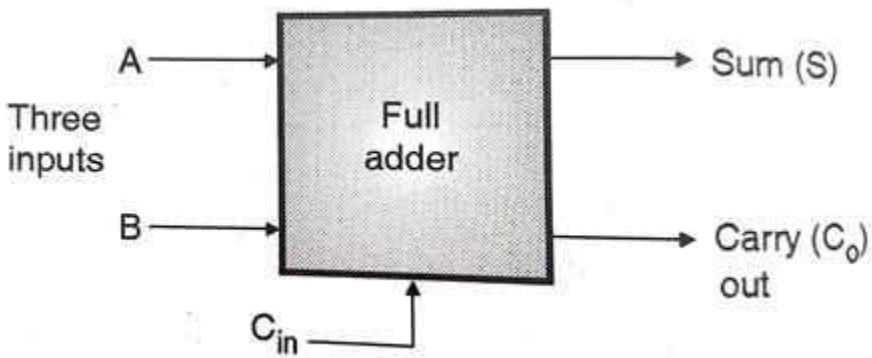
$S = A\bar{B}$

$C = AB$

(b) Truth table

Full Adder

- Full Adder is a combinational circuit that performs the addition of three bits (two significant bits and previous carry).
- It consists of three inputs and two outputs, two inputs are the bits to be added, the third input represents the carry form the previous position.
- The full adder is usually a component in a cascade of adders, which add 8, 16, etc, binary numbers.
- The output is equal to 1 when only one input is equal to 1 or when all three inputs are equal to 1.
- The output has a carry 1 if two or three inputs are equal to 1.



(a) Block Diagram

Inputs			Outputs	
A	B	C_{in}	S	C_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$S = \bar{A}\,\bar{B}\,C_{in}$

$S = \bar{A}\,B\,\bar{C}_{in}$

$C_o = \bar{A}\,B\,C_{in}$

$S = A\,\bar{B}\,\bar{C}_{in}$

$C_o = A\,\bar{B}\,C_{in}$

$C_o = A\,B\,\bar{C}_{in}$

$S = A\,B\,C_{in}$

$C_o = A\,B\,C_{in}$

(b) Truth table

Full Adder

Expression for sum output :

$$S = \overline{A}\,\overline{B}C_{in} + \overline{A}BC_{in} + ABC_{in} + A\overline{B}C_{in}$$

$$S = C_{in}\underbrace{(\overline{A}\,\overline{B} + AB)}_{\text{EX-NOR}} + \overline{C}_{in}\underbrace{(\overline{A}B + A\overline{B})}_{\text{EX-OR}}$$

(C-6375)

$$\therefore\ S = C_{in}\,\overline{(\overline{A}B + A\overline{B})} + \overline{C}_{in}\,(\overline{A}B + A\overline{B})$$

$$\text{Let } X = \overline{A}B + A\overline{B}$$

$$\therefore\ S = C_{in}\overline{X} + \overline{C}_{in}X = C_{in} \oplus X$$

$$\therefore\ S = C_{in} \oplus (\overline{A}B + A\overline{B})$$

$$\text{But } \overline{A}B + A\overline{B} = A \oplus B$$

$$\therefore\ S = C_{in} \oplus A \oplus B$$

Expression for carry output :

$$C_o = \bar{A}BC_{in} + A\bar{B}C_{in} + AB\bar{C}_{in} + ABC_{in}$$

$$C_o = (\bar{A}B + A\bar{B})\, C_{in} + AB\,(\underbrace{\bar{C}_{in} + C_{in}}_{1}) = \bar{A}BC_{in} + A\bar{B}C_{in} + AB$$

$$= \bar{A}BC_{in} + A\bar{B}C_{in} + AB\,(\underbrace{1 + C_{in}}_{1})$$

$$= \bar{A}BC_{in} + A\bar{B}C_{in} + AB + ABC_{in}$$

$$= BC_{in}\,(\underbrace{\bar{A} + A}_{1}) + A\bar{B}C_{in} + AB$$

$$= BC_{in} + A\bar{B}C_{in} + AB$$

$$= BC_{in} + A\bar{B}C_{in} + AB\,(\underbrace{1 + C_{in}}_{1})$$

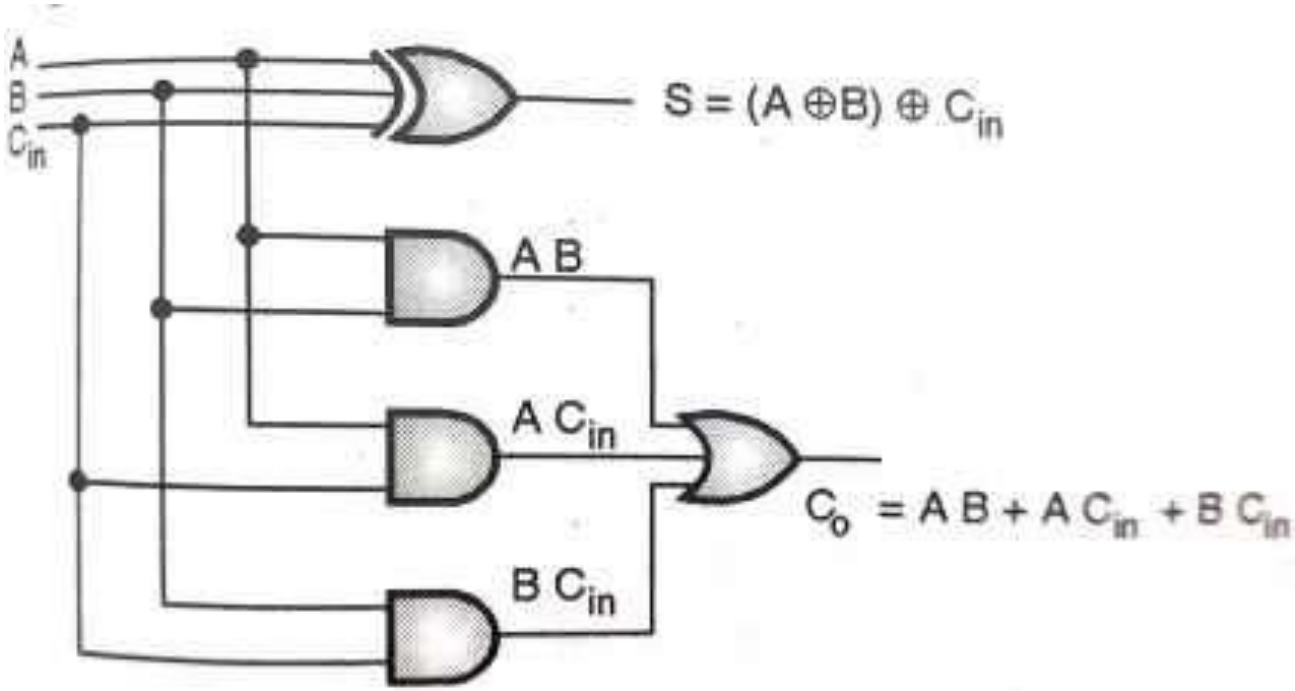
$$= BC_{in} + A\bar{B}C_{in} + AB + ABC_{in}$$

$$= BC_{in} + AB + AC_{in}\,(\underbrace{\bar{B} + B}_{1})$$

$$C_o = AB + AC_{in} + BC_{in}$$

Full Adder

- The implementation of half adder using exclusive–OR and an AND gates is used
- The inputs to the XOR gate are also the inputs to the AND gate.



(C) Logic Diagram for Full Adder

Categories of Circuits

- With the two general categories of circuits being:

1. Combinational Circuits where the input values explicitly determine the output
2. Sequential Circuits where the output is a function of the input values as well as the existing stage of the circuit
    - a. Thus, sequential circuits usually involve the storage of information

1-bit Memory Cell: Flip-flops

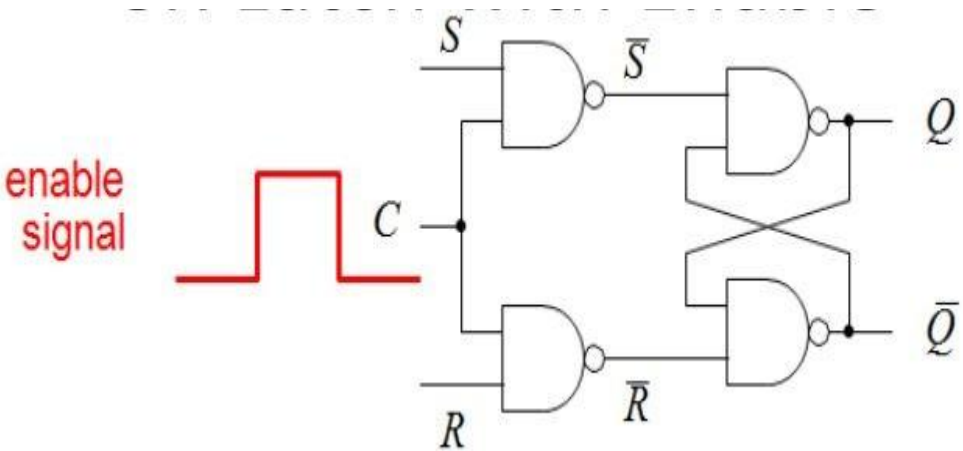
- A latch is an electronic logic circuit that has two inputs and one output.
- Stores a data value
  - non-volatile; will not lose value over time
  - often incorporated in static memory
- Technically a latch is level triggered and a flip-flop is edge triggered (on the clock cycle).
  - SR or set–reset latch, which may also be called a SR flip-flop
  - D or data flip-flip
  - T or toggle flip-flop
  - JK flip-flop
- **MEMORY**
- We know that flip-flop can store either zero or one permanently until a change is made in the inputs.
- Flip-flops are the basic technology behind computer memory chips

S-R Flip-flop

- Then the SR description stands for "Set-Reset".
- The simple flip-flop is basically a one-bit memory bistable device that has two inputs, one which will "SET" the device (meaning the output = "1"), and is labelled S and one which will "RESET" the device (meaning the output = "0"), labelled R.
- S-R Latch Acts as a simple memory with two states
  - The latch - holds (stores) when S = R = 0
  - is set (to 1) by bringing S = 1 with R = 0
  - is reset (to 0) by bringing R = 1 with S = 0
- The condition S = R = 1 must be avoided because it leads to an indeterminate condition.
- This can cause a **race** condition to occur.

SR Flip-flop

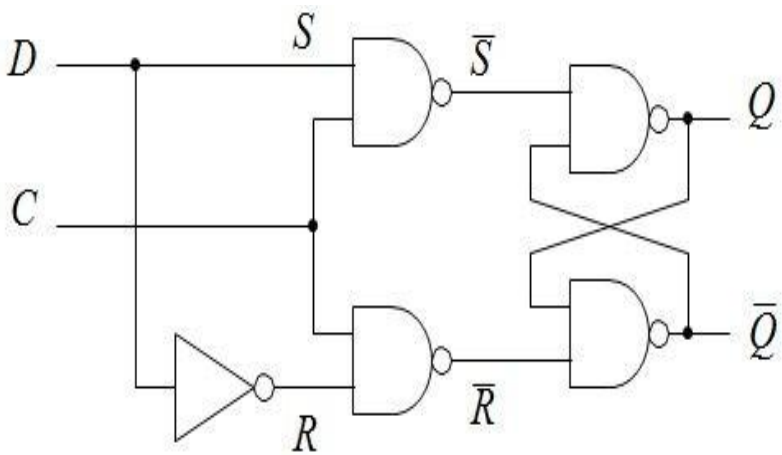
- The S and R inputs only effect the output states when the clock input is high.
- The flip-flop holds (stores) its value while the enable input is low — latches it! (no change in previous state and next state)
- Any changes in the inputs during the time when enable is high will affect the output immediately
- This circuit still has a major problem: the stored value is indeterminate if S = R = 1 when th



S	R	C	Q_n	Q_{n+1}	description
0	0	↓	0	0	hold
0	0		1	1	No Change
0	1	↓	0	0	clear (reset)
0	1		1	0	
1	0	↓	0	1	set
1	0		1	1	
1	1	↓	0	?	indeterminate
1	1		1	?	— avoid Race condition

D Flip-Flop

- The problem with S = R = 1 can be avoided using a common input D as shown.
- It is a variant of the more basic S-R flip-flop (**Set-Reset)**
- The output of the flip-flop now: –
  - **follows the D input** while C = 1 (transparent)
  - holds its value while C = 0 (Q = last Q when C went low) no matter what happens at the input
- This circuit is often called a transparent latch.



D	C	Q_n	Q_{n+1}	description
0	↑	0	0	Clear (reset)
0		1	0	
1	↑	0	1	Set
1		1	1	

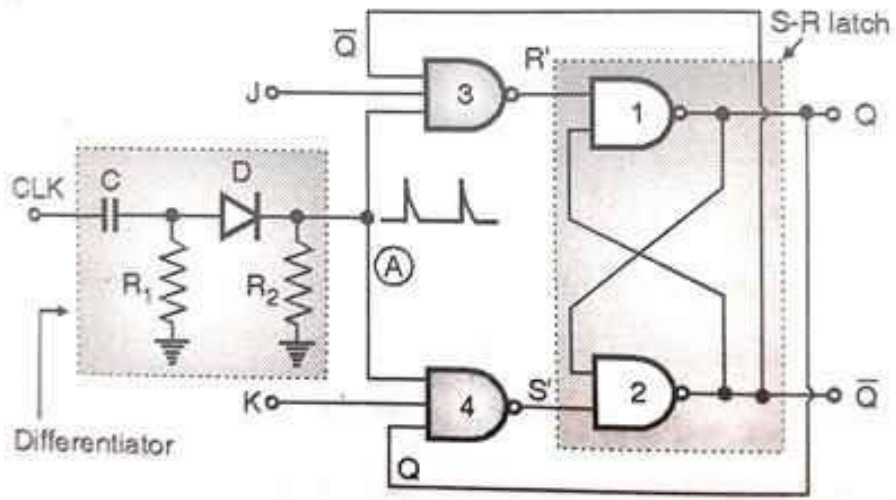
D Flip-Flop

- It acts as a simple memory or ‘latch’.
- The previous value stored, Qn, has no effect, unlike other flip-flops. It therefore acts as a simple memory or ‘latch’.
- The most widely used flip-flops: simple to build and design with.
- A register comprises several D flip-flops, one for each bit to be stored.

JK Flip-Flop

The name JK flip-flop is termed from the inventor Jack Kilby from texas instruments.
One of the most useful and versatile flip-flop
The fundamental difference of this device is the feedback paths to the NAND gates of the input.

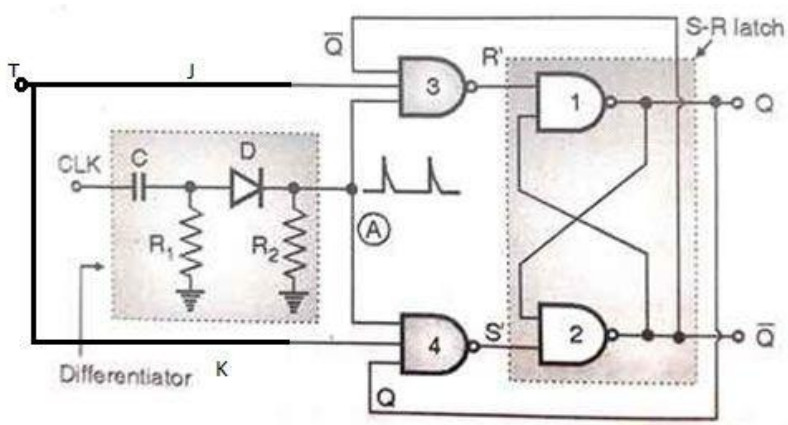
1. if the J and K input are both at 1 and clock is applied then the output will change/toggle irrespective of its previous condition.
2. if the J and K input are both at 0 and clock is applied then the output will not change.
3. When J=0 and K=1 output is 0 (Reset condition) and when J=1 and K=0 then output is 1 (Set condition)
3. No indeterminate condition, in the operation of JK flip-flop.



Case No.	Inputs			Outputs		State
	CLK	J	K	Q_{n+1}	$\overline{Q}_{n+1}$	
Case I	0 or 1	x	x	No change	No change	Flip flop is disabled
Case II	↓	x	x	No change	No change	
Case III	↑	0	0	No change	No change	
Case IV	↑	0	1	0	1	Reset
Case V	↑	1	0	1	0	Set
Case VI	↑	1	1	$\overline{Q}_n$	Q_n	Toggle

Toggle (T) Flip-Flop

- The T flip-flop is a single input version of the JK flip-flop.
- The T flip-flop is obtained from the JK type if both inputs are tied together.
- The toggle, or T, flip-flop is a bistable device, where the output of the T flip-flop "toggles" with each clock pulse.
- Till CP=0, the output is in hold state (three input AND gate principle).
- When CP=1, for T=0, previous output is memorized by the circuit.
  When T=1 along with the clock pulse, the output toggles from the previous value

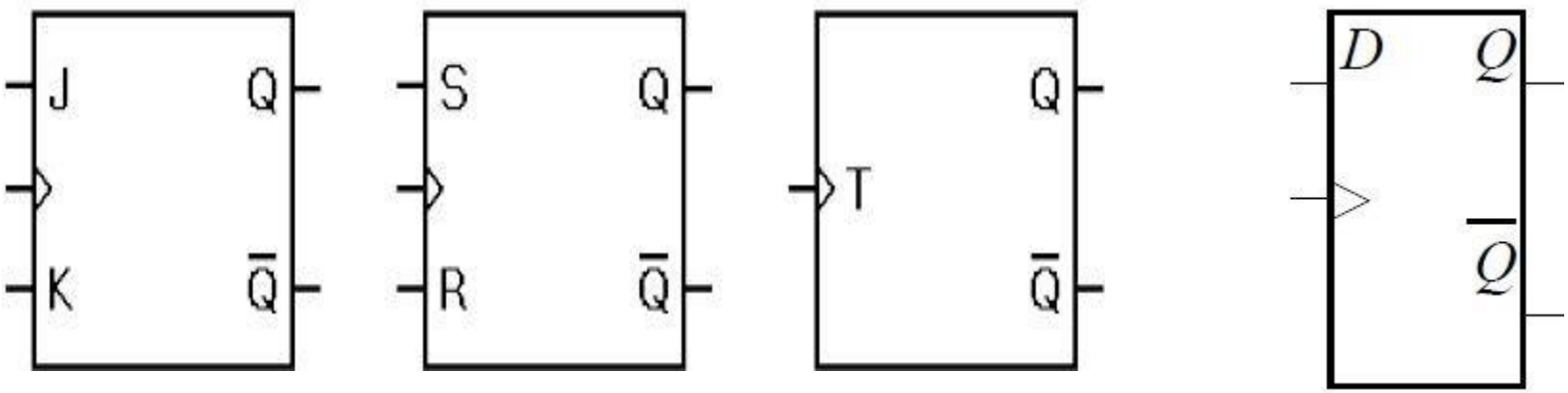


Inputs		Outputs		State
CLK	T	Q_{n+1}	$\bar{Q}_{n+1}$	
↑	0	Q_n	$\bar{Q}_n$	No Change
↓	x	Q_n	$\bar{Q}_n$	No Change
1	x	Q_n	$\bar{Q}_n$	No Change
0	x	Q_n	$\bar{Q}_n$	No Change
↑	1	$\bar{Q}_n$	Q_n	Toggle

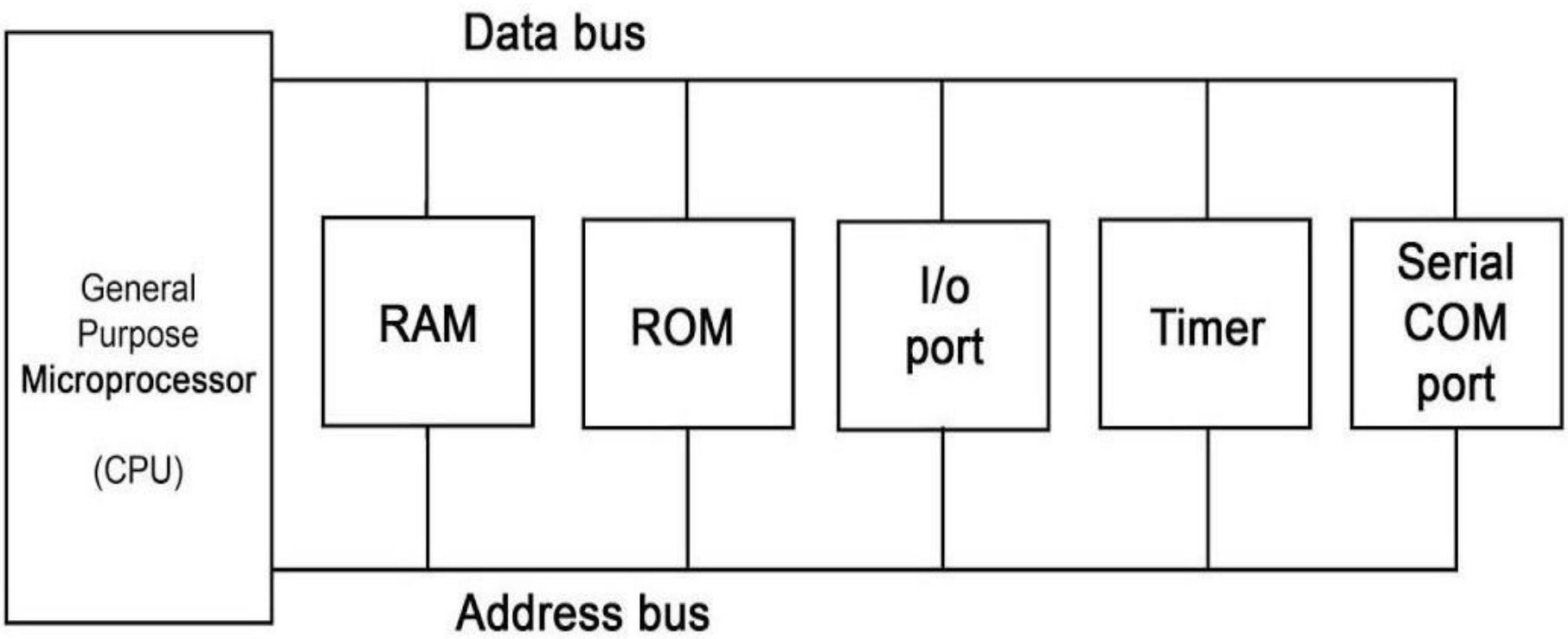
Flip-flop disabled

Flip-Flop Symbols and Applications

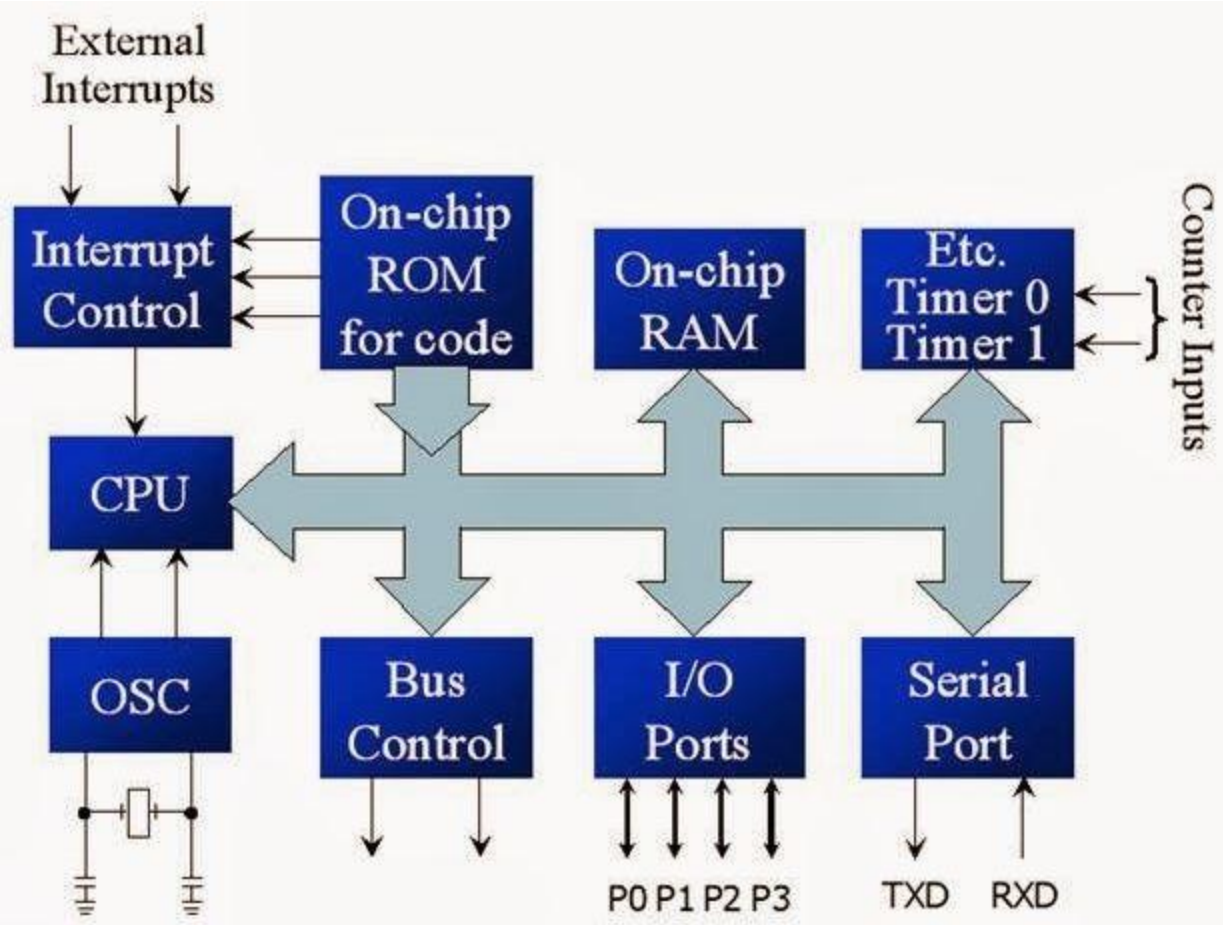
- Typical flip-flop symbols, using a “black-box approach”, an approach that is useful to represent specific clusters of electronic components .
- There are two general ways in which flip-flops may be interconnected to perform useful functions: *counters and registers*



Block Diagram of Microprocessor



Block Diagram of Microcontroller



Microcontroller

A microcontroller basically contains one or more following components:

- Central processing unit(CPU)
- Random Access Memory)(RAM)
- Read Only Memory(ROM)
- Input/output ports
- Timers and Counters
- Interrupt Controls
- Analog to digital converters
- Digital to analog converters
- Serial interfacing ports
- Oscillatory circuits

Microcontrollers

- Microcontroller internally consists of all features required for a computing system and functions as a computer without adding any external digital parts in it.

CPU:

- CPU is the brain of a microcontroller. CPU is responsible for fetching the instruction, decodes it, then finally executed. CPU connects every part of a microcontroller into a single system. The primary function of CPU is fetching and decoding instructions. The instruction fetched from program memory must be decoded by the CPU.

Microcontrollers

- **Memory:** Function of memory in a microcontroller is the same as a microprocessor. It is used to store data and program. A microcontroller usually has a certain amount of RAM and ROM (EEPROM, EPROM, etc) or flash memories for storing program source codes.

- **I/O ports:** Parallel input/output ports are mainly used to drive/interface various devices such as LCD'S, LED'S, printers, memories, etc. to a microcontroller.

- **Serial Ports:** Serial ports provide various serial interfaces between a microcontroller and other peripherals like parallel ports.

Microcontrollers

Timers and counters:

- The timers and counters provide all timing and counting functions inside the microcontroller. The major operations of this section are performed clock functions, modulations, pulse generations, frequency measuring, making oscillations, etc. This also can be used for counting external pulses.

A to D converter:

- ADC converters are used for converting the analog signal to digital form. The input signal in this converter should be in analog form (e.g. sensor output) and the output from this unit is in digital form. The digital output can be used for various digital applications (e.g. measurement devices).

Microcontrollers

D to A converter:

- DAC perform reversal operation of ADC conversion, converts the digital signal into analog format. It usually used for controlling analog devices like DC motors, various drives, etc.

Interrupt Control:

- The interrupt control used for providing interrupt (delay) for a working program. The interrupt may be external (activated by using interrupt pin) or internal (by using interrupt instruction during programming)

Oscillatory Circuits:

- Quartz crystal oscillator used to generate the stable oscillations needed for the clock pulses ,required for synchronous operation of the microcontrollers.

Difference Between Microprocessor and Microcontroller

Sr No	Microprocessor	Microcontroller
1	It is a central processing unit on a single silicon based integrated chip	It is a single chip computer
2	It has no Memory, IO Units, Timers and other peripheral on the chips.	It has a CPU along with Memory, IO Units, Timers and other peripherals embedded on the single chip.
3	It uses an external bus to interface Memory, IO Units, Timers and other peripherals.	It uses an internal controlling bus which is not available to the board designer.
4	It is used for general purpose applications which are able to handle loads of data e.g. in PCs, laptops etc.	It is used for application specific systems such as washing machine, remote control, microwave oven etc.
5	It is complex and expensive with large number of instructions to process.	It is simple and inexpensive with less number of instructions to process.
6	Examples of microprocessor Ics are 8085,8086,80x86, Pentium etc	e.g. 8051, PIC, ARM

Text and Reference Books

- “Modern Digital Electronics” by R.P. Jain, 4th Edition, Tata McGraw Hill
- “Digital Fundamentals” by Thomas. L. Floyd, 11th Edition, Pearson

Thank You!!!